Snackautomat Challenge

Dario Züger, Fidan Fetahi, Kaylee Osagie

NOSER YOUNG AG

Inhaltsverzeichnis

Einleitung 1
1. Informieren (I) 1
2. Planen (P) 2
2.1 Arbeitsaufteilung 3
3. Entscheiden (E) 3
4. Realisieren (R) 3
5. Kontrollieren (K) 3
6. Auswerten (A) / Reflexion 4

Einleitung

Im Rahmen der Snackautomat-Challenge von Noser Young haben wir einen digitalen Simulator für einen Verkaufsautomaten in Java entwickelt. Ziel des Projekts war es, eine robuste Applikation zu erstellen, die sowohl kundenorientierte Prozesse (Produktauswahl, Bezahlung) als auch administrative Wartungsaufgaben (Lagerverwaltung, Preisänderung) abbildet. Ein besonderes Augenmerk lag dabei auf der sauberen Trennung von Benutzeroberfläche und Geschäftslogik nach dem EVA-Prinzip, um eine hohe Code-Qualität und Fehlerresistenz zu gewährleisten.

1. Informieren (I)

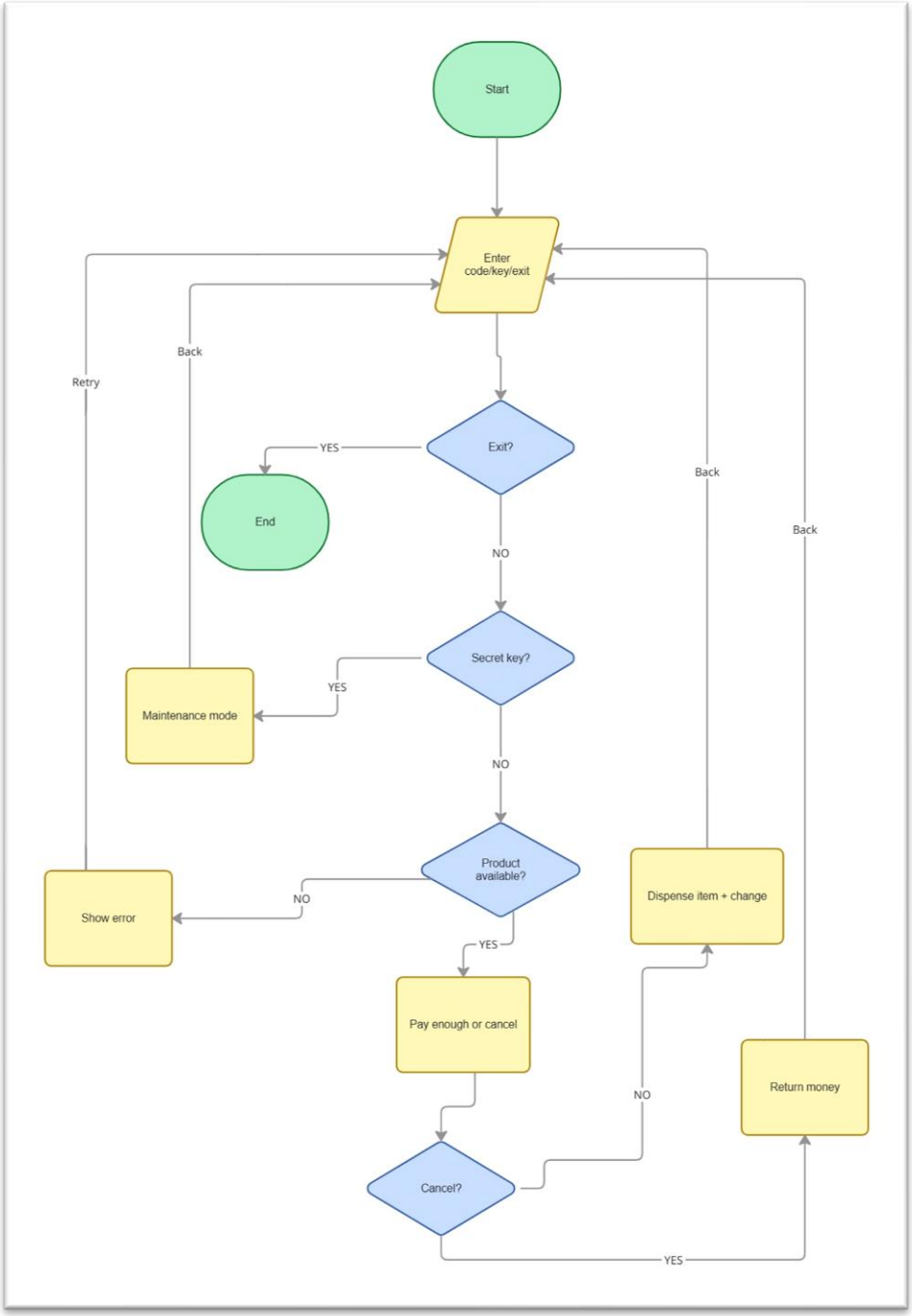
In dieser Phase haben wir den Projektauftrag analysiert.

- **Kernpunkte:** Modellierung eines Snackautomaten mit Kauf- und Wartungsfunktionen.
- **Besonderer Fokus:** Strikte Trennung von Business-Logik und User-Interaktion (EVA-Prinzip).
- **Analyse der Features:** Geld einwerfen, Produktauswahl, Rückgeld-Logik, Abbruchfunktion sowie passwortgeschützte Wartung (Auffüllen, Ersetzen, Preisänderung).

2. Planen (P)

Hier haben wir die theoretische Grundlage geschaffen:

- **UML-Diagramm:** Erstellung eines Klassendiagramms zur Festlegung der Struktur (Product, VendingMachine, Simulation, Starter).
- **Fehlerszenarien:** Antizipation von Problemen wie unzureichendem Guthaben, leerem Lagerbestand oder ungültigen Eingaben.
- **Ablaufplanung:** Festlegung der Kaufsequenz (Geld -> Wahl) und des Wartungs-Logins.



2.1 Arbeitsaufteilung

Name	Aufgaben
Fidan	• Entwicklung der Product-Klasse • Mitarbeit an der VendingMachine • Unterstützung bei Präsentation (PPT) und Dokumentation
Kaylee	• Entwicklung der Simulation-Klasse (UI / Ablauf) • Fehlerbehebung (Bugfixes)• Erstellung der Präsentation (PPT)
Dario	• Entwicklung der VendingMachine (Core-Logik) • Erstellung der Dokumentation

3. Entscheiden (E)

Wir haben uns im Team auf folgende Standards geeinigt:

Technologie: Java als objektorientierte Basis.

Architektur: Die VendingMachine fungiert als zentrale Instanz für Daten und Logik.

Validierungs-Strategie: Wir haben uns entschieden, ungültige Parameter (z.B. negative Beträge) konsequent mit Exceptions (IllegalArgumentException) abzufangen, anstatt nur Fehlertexte auszugeben. Dies garantiert eine hohe Datensicherheit.

4. Realisieren (R)

Während der Realisierung haben wir die Kernfunktionen implementiert und dabei folgende technische Herausforderungen gelöst:

- **Case-Insensitivity:** Optimierung der Benutzereingaben mittels .toUpperCase(), um die Bedienbarkeit zu erhöhen (z.B. akzeptiert das System nun "a3" und "A3").
- **Gleitkomma-Präzision:** Da Computer bei double oder float Rechenungenauigkeiten aufweisen (z.B. 1.99999998 statt 2.0), haben wir die Logik so angepasst, dass Rundungsfehler abgefangen werden.

5. Kontrollieren (K)

In der Testphase haben wir das System auf Herz und Nieren geprüft. Dabei identifizierten wir kritische Logikfehler, die wir im Refactoring behoben haben:

- **Initialer Status:** Wir stellten fest, dass der Automat im Leerzustand keine Produktauswahl erlauben darf. Das bedeutete, dass es nicht kundenfreundlich war, deshalb mussten wir das ändern
- **Transparenz:** Es fehlte eine Statusanzeige. Wir haben deshalb einen Geld-Status eingebaut, die dem Kunden nach jedem Münzeinwurf den aktuellen Kontostand anzeigt.
- **Kapazitätsgrenzen:** Beim refill haben wir analysiert, dass ein unendlicher Lagerbestand unrealistisch ist. Deshalb mussten wir das noch begrenzen

6. Auswerten (A) / Reflexion

Rückblickend war die Entscheidung, das EVA-Prinzip strikt einzuhalten, der Schlüssel zum Erfolg. Besonders lehrreich war die Erkenntnis über die mathematischen Grenzen von Datentypen (double-Problematik). Wir haben gelernt, dass Planung (UML) zwar wichtig ist, aber erst der intensive Test am 'lebenden Objekt' die wahren UX-Probleme (wie die Case-Sensitivity) aufdeckt. Als Team haben wir die Deep-Work-Phasen effizient genutzt, um die komplexe Validierungslogik in der VendingMachine fertigzustellen, ohne uns gegenseitig zu unterbrechen.